

Ansible



Ansible: Automating the Modern Infrastructure.





Introduction

Scenario: Academic lab setting

Task: Configure a Nginx web server for a project.

Scale: 1 machine.

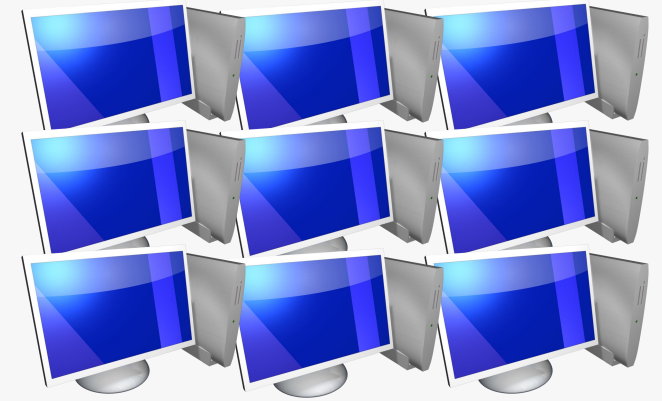
➔ **Manual configuration (SSH, typing commands)** is perfectly fine.



Scenario: Professional enterprise setting

Task: Configure Nginx web servers for a big application.

Scale: Dozens or hundreds of servers.



The issues with manual setup:

- **Wasted time & effort:** Takes hours/days to repeat steps.
- **Run Manually:** SSH into 100+ servers.
- **Human error:** High risk of typos or skipping steps.
- **Inconsistency:** Some servers require slightly different setups.

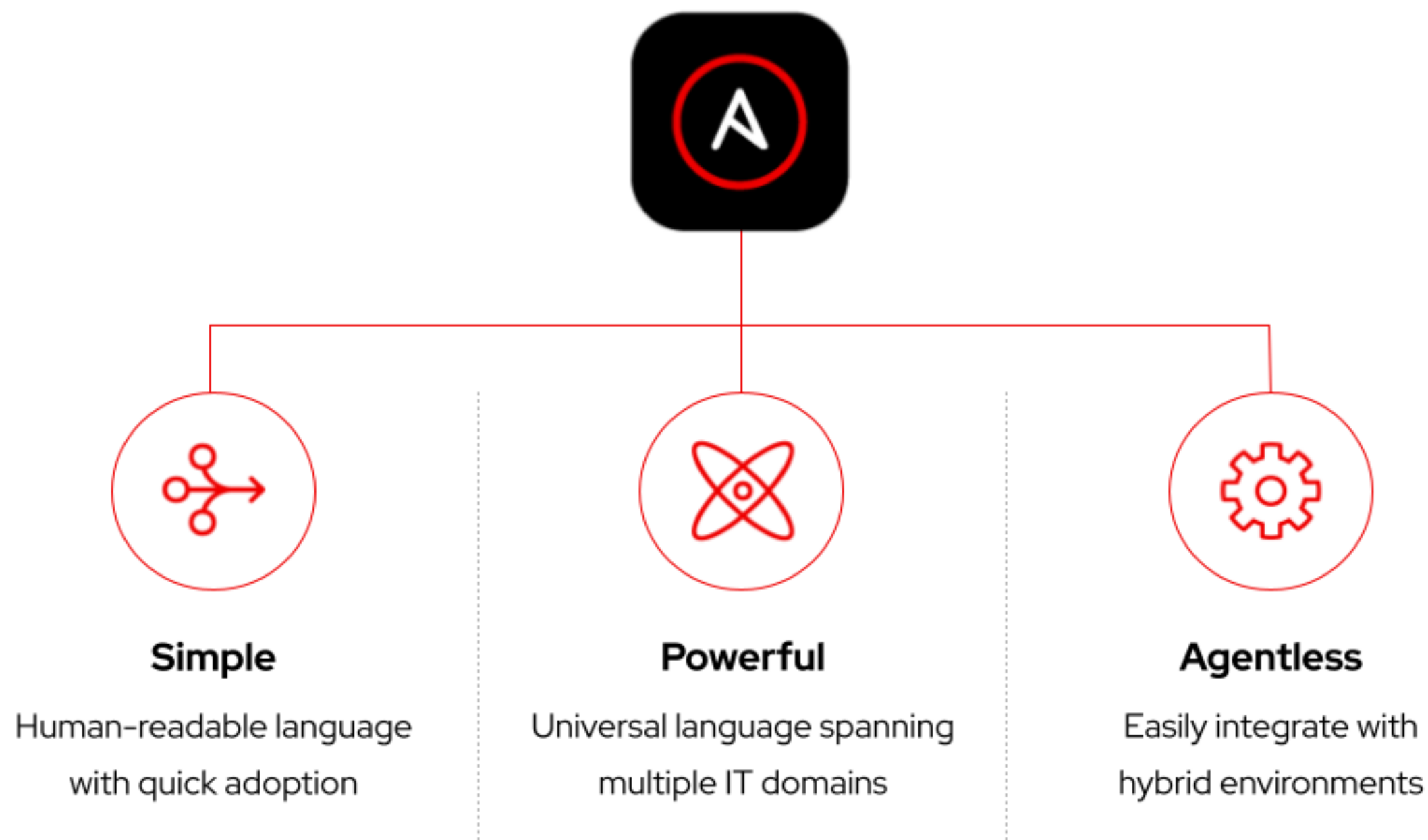


➔ An **automation tool** is needed to handle the scale.

What is Ansible?

- **Ansible** is an **open-source automation and configuration management tool by Red Hat**.
- It allows you to define your system configurations **as code** ("infrastructure as code").
- Created by Michael DeHaan and officially **released in 2012**.
- It quickly gained popularity by eliminating common automation headaches:
 - Agentless: Uses standard SSH, no extra software needed on servers.
 - Tasks could be written in straightforward YAML files rather than complicated programming languages.





How Ansible solves the problem?

The Task: Deploy a Java Spring Boot environment on 10 servers.

The Manual Process Way:

- **Connect** `ssh user@server-1...10`
- Type commands (`apt update`, `apt install openjdk-17-jdk`, ...)
- Exit.
- Repeat for Server 2, Server 3... Server 10.
- **The Pain Points:** Time-consuming, prone to typos, impossible to scale, inconsistent.

The Ansible Way: Automation through YAML.

- Write **ONE** human-readable **yaml** file.
- Run **ONE** command: `ansible-playbook setup-java.yaml`
- Ansible connects via SSH to all 10 servers **in parallel** and does the work for you.
- **The Result:** Consistent, error-free, and scalable to 10,000 servers.



```
ec2-user@ip-172-31-30-233:~  
---  
- name: Install HTTPD using Ansible Playbook  
  hosts: slave-1  
  become: true  
  
  tasks:  
    - name: Install HTTPD package  
      package:  
        name: httpd  
        state: present  
  
    - name: Start HTTPD service  
      service:  
        name: httpd  
        state: started  
        enabled: yes
```



Terminology & Core Concepts

Core concepts — The infrastructure

CONTROL NODE

- Install the Ansible package.
- The machine where **Ansible is installed** and **automation is executed**.

SSH / WinRM



MANAGED NODES

- Also referred to as '**hosts**'.
- These are the **target devices** (servers, network appliances) you **aim to manage**.

Core concepts — Agentless architecture

Ansible **does not require an agent** on managed nodes. It leverages standard, pre-existing administrative protocols:

- **Linux/Unix:** connects via standard **OpenSSH** by default, most Ansible modules use **Python** on the managed node.
- **Windows:** connects via **WinRM** or **SSH**; uses **PowerShell** to execute the module.

Advantages:

- **No** persistent agent to install or upgrade.
- **No** background Ansible agent running on managed hosts.
- **Easier onboarding** of new hosts.

Core concepts — The inventory (1)

An **inventory** defines the **managed hosts and groups** that Ansible can target.

Common formats: **INI** and **YAML**.

An inventory can define:

- **Managed nodes.**
- **Groups** of hosts.
- Host/group **variables** such as connection addresses or users.

Example 1 (INI Format)

```
[webservers]
web1.example.com
web2.example.com

[dbservers]
db1.example.com
db2.example.com
```

Example 2 (YAML Format)

```
webservers:
  hosts:
    web1.example.com:
    web2.example.com:

dbservers:
  hosts:
    db1.example.com:
    db2.example.com:
```

Core concepts — The inventory (2)

Two implicit groups always exist:

- **all:** every host in the inventory.
- **ungrouped:** contains hosts that do **not** belong to another group.

[all:vars] sets a variable applied to every host in the inventory.

Groups can nest via **:children**, forming broader groups from smaller ones.

```
[webservers]
web1.example.com
```

```
[dbservers]
db1.example.com
```

```
[all:vars]
ansible_user=ubuntu
```

```
[production:children]
webservers
dbservers
```

Core concepts — Modules

The code or binaries that Ansible copies to and executes on each managed node to perform one specific job.

Comes from `ansible.builtin` (built in) or from collections (like `community.docker`).

Start Docker
Container

Install a Package

Copy a File

...

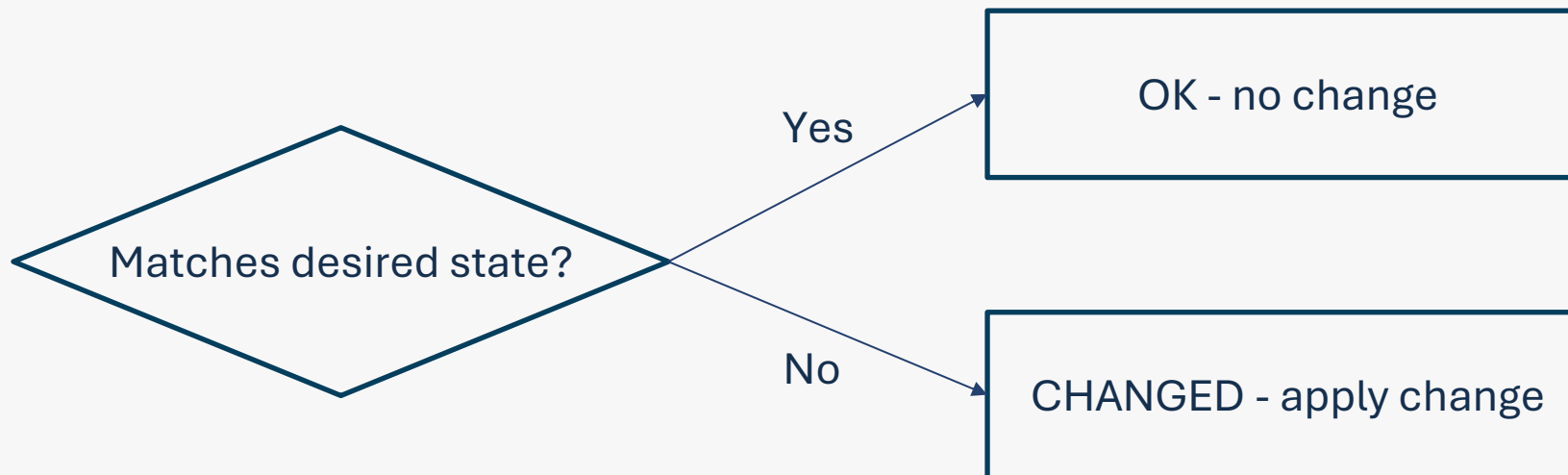
...

Module Index

- [All modules](#)
- [Cloud modules](#)
- [Clustering modules](#)
- [Commands modules](#)
- [Crypto modules](#)
- [Database modules](#)
- [Files modules](#)
- [Identity modules](#)
- [Inventory modules](#)
- [Messaging modules](#)
- [Monitoring modules](#)
- [Net Tools modules](#)
- [Network modules](#)
- [Notification modules](#)
- [Packaging modules](#)
- [Remote Management modules](#)
- [Source Control modules](#)
- [Storage modules](#)
- [System modules](#)
- [Utilities modules](#)
- [Web Infrastructure modules](#)
- [Windows modules](#)

Operational philosophy — Idempotency

- **Idempotency** means running the same automation **repeatedly** should normally **produce the same final state**.
- Most Ansible modules are **designed to be idempotent** (exception: command/shell/raw)
- Before executing a change, the module **checks the current state** of the remote system.
- If the system is already in the desired state, Ansible **safely exits** the task without making any redundant modifications.



Core concepts — The execution hierarchy

PLAYBOOKS

- Files in **YAML format**, that Ansible uses to **deploy and configure managed nodes**.
- Contain one or more **Plays**.

PLAYS

An **ordered list of tasks mapped to managed nodes** in an inventory.

TASKS

Define an action to apply to managed hosts, usually by invoking a module with specific arguments. Runs in order, top to bottom.

HANDLERS

- A **special form of a Task** that **only executes when notified** by a regular task which resulted in a 'changed' status.
- A handler runs at most once per play, at the end by default.

Core concepts — Playbook Example

```
- name: Install Nginx                                # play: targets the webservers group
  hosts: webservers
  become: true

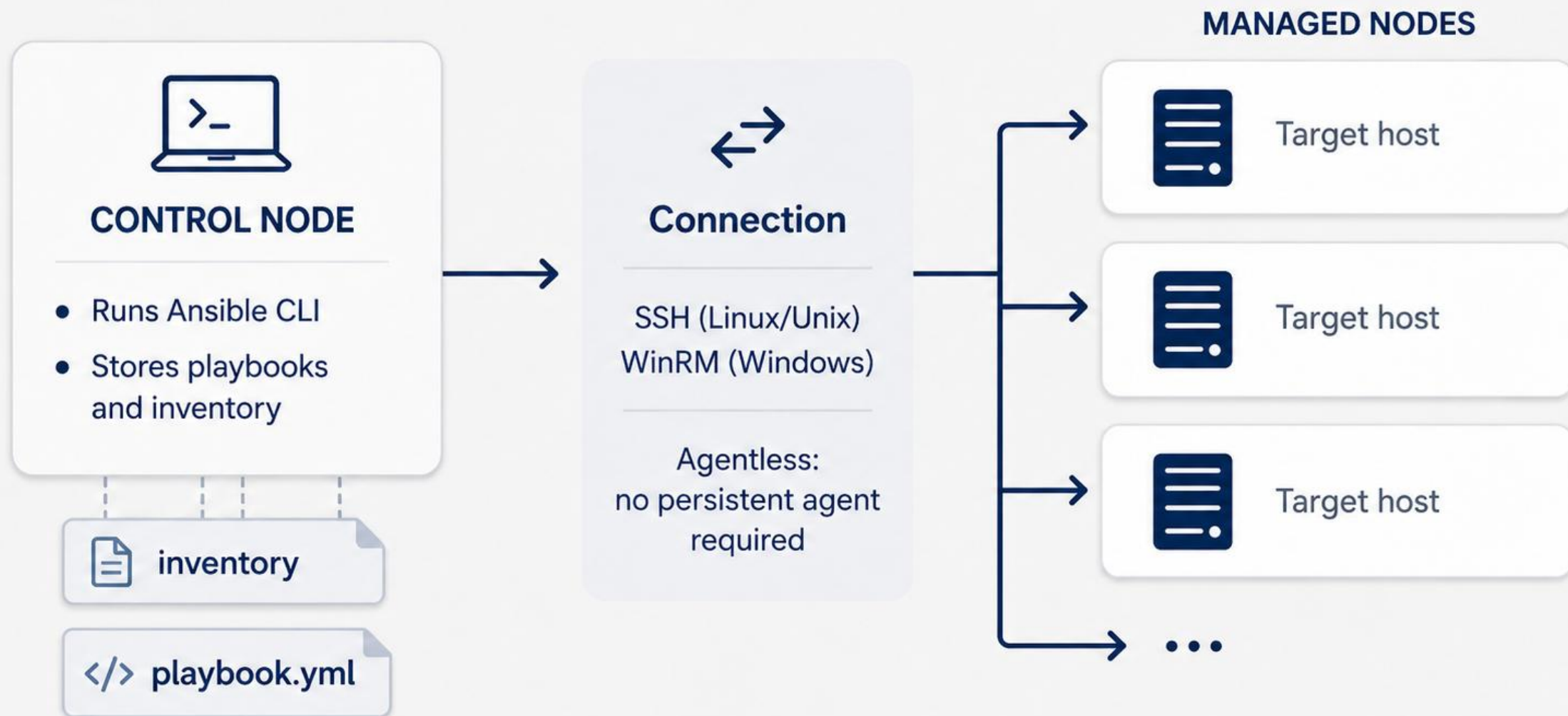
  tasks:
    - name: Install package                          # task 1: install nginx
      ansible.builtin.apt:                          # module: apt
        name: nginx
        state: present

    - name: Deploy config file                       # task 2: copy config file
      ansible.builtin.copy:                         # module: copy
        src: nginx.conf
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx                        # notify: only fires if this task changes something

    - name: Start service                           # task 3: start nginx
      ansible.builtin.service:                     # module: service
        name: nginx
        state: started
        enabled: true

  handlers:
    - name: Restart nginx                          # handler: only runs if notified above
      ansible.builtin.service:                     # module: service
        name: nginx
        state: restarted
```


Architecture



Advance concepts

ROLES

A well-written role is reusable, testable, and easy for your team to understand.

JINJA2

A templating engine used by Ansible to create dynamic content.

ANSIBLE GALAXY & COLLECTIONS

A community-driven platform where users can share pre-built Ansible roles

VAULT

Providing a way to encrypt and manage sensitive data.

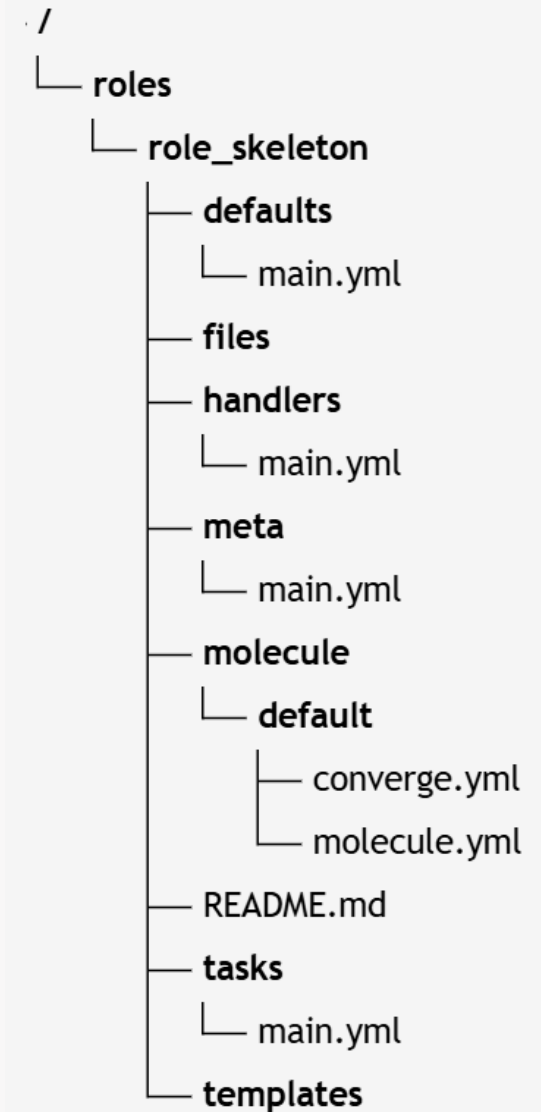
ROLES

What is?

Role is a structured, reusable way to organize playbooks and related automation assets.

Instead of writing a single, massive playbook file, Ansible Roles allow you to break down your automation logic into a standardized folder hierarchy containing tasks, variables, handlers, templates, and documentation.

Structure



Ansible Galaxy & Collections

Ansible Galaxy serves as a central repository for Ansible content. It contains:

- **Ansible Roles:** Pre-packaged automation for specific tasks or configurations.
- **Ansible Collections:** Introduced in Ansible 2.9+, these are bigger bundles. They can hold multiple roles, modules, plugins, and documentation.

Ansible Collections are the newer, standard way to package Ansible content.

Collections use a <namespace>.<collection_name> format such as:

cisco.ios or **junipernetworks.junos**

Example:

```
ansible-galaxy collection install cisco.ios
```

JINJA2

Optimize Ansible Deployments

Ansible uses Jinja2 templating to enable dynamic expressions and access to **variables** and **facts**

Jinja2 extends **Ansible variables** by allowing:

- Variable substitution inside files
- Conditional rendering (if/else)
- Iteration using loops (for)
- Using filters (default, upper, lower, etc.)
- Generating different configurations for different servers/environments.

Jinja2 Templating

- Define src, dest, and mode
- backup: true for safe rollbacks
- Set fallbacks with default()
- Iterate items using for loops
- Format data with upper & join
- Apply if / elif / else conditions
- Leverage live ansible_facts
- Fetch remote values via hostvars
- Clean layout with whitespace control (-%})
- Use {% raw %} block tags
- Validate syntax before saving (validate)
- Trigger updates via notify handlers
- Preview changes with --check --diff

Managing vault passwords

Choosing between a single password and multiple passwords

Depending on your needs, you might want a different password for each encrypted file, for each directory, or each environment.

Managing multiple passwords with vault IDs

- Pass it with `--vault-id` to the `ansible-vault` command when you create encrypted content
- Include it wherever you store the password for that vault ID
- Pass it with `--vault-id` to the `ansible-playbook` command when you run a playbook that uses content you encrypted with that vault ID

Storing and accessing vault passwords

Storing passwords in files

Storing passwords in third-party tools with vault password client scripts

Encrypting content with Ansible Vault

Creating encrypted variables

```
ansible-vault encrypt_string --vault-password-file a_password_file 'foobar'  
name 'the_secret'
```

The command above creates this content:

```
the_secret: !vault |
```

```
$ANSIBLE_VAULT;1.1;AES256 623133653966623430613934643361633837643  
73764613633653634306231386433626436623361613433366535396636353433  
3632666535333761666131620a663537646436643839616531643561633962653  
33966386166373632626539326166353965363262633030333630313338646335  
303630438626666666137650a3536386434356666336339643663386330666232  
3461643237323133333316564
```



Extension & Conclusion

What if?

You don't like CLI?

AWX & AAP

AWX: open-source web UI / API for Ansible

AAP: Enterprise platform by Red Hat



Not only run but manage:

Jobs

Inventory

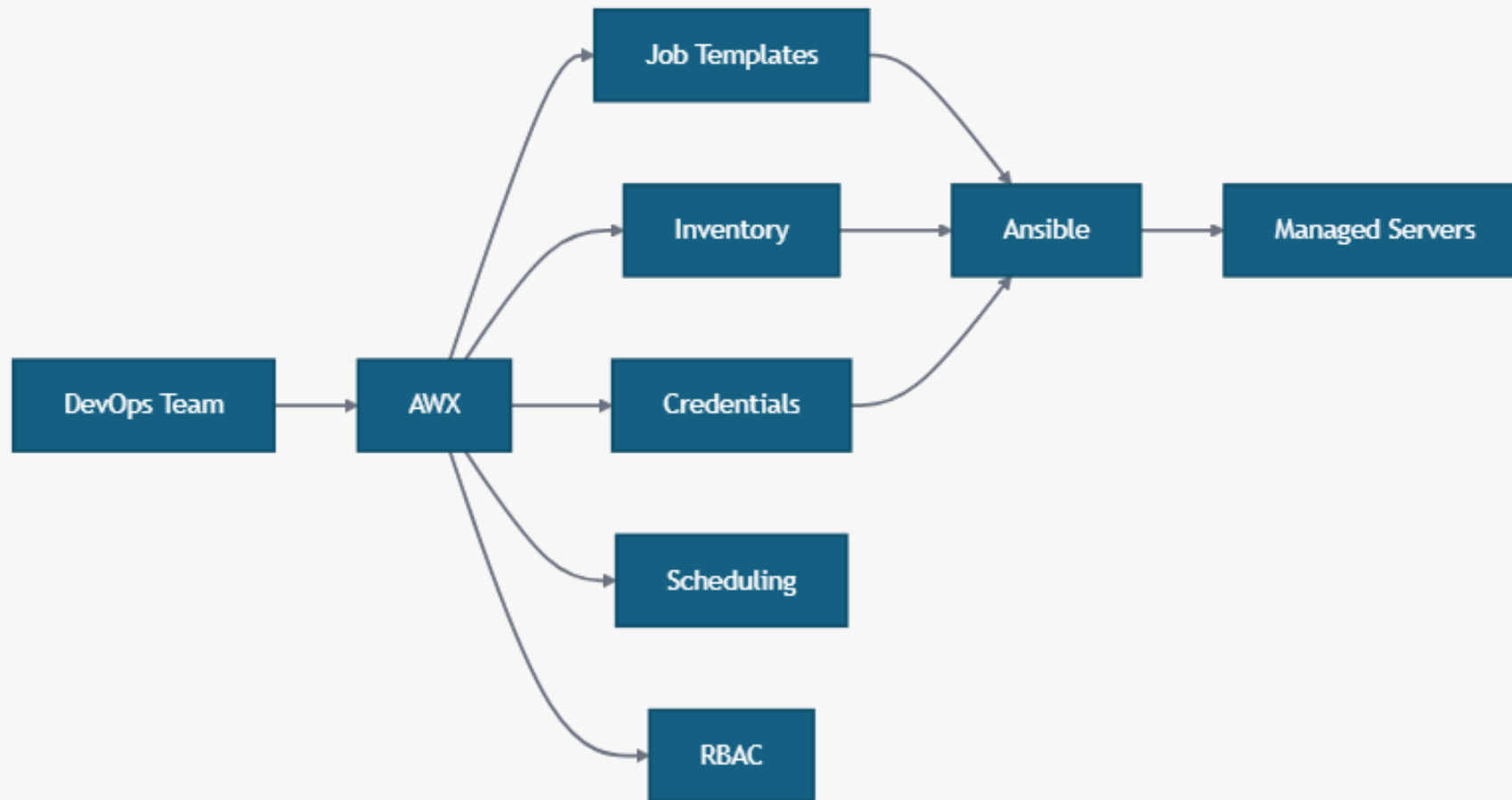
Log/history

Credentials

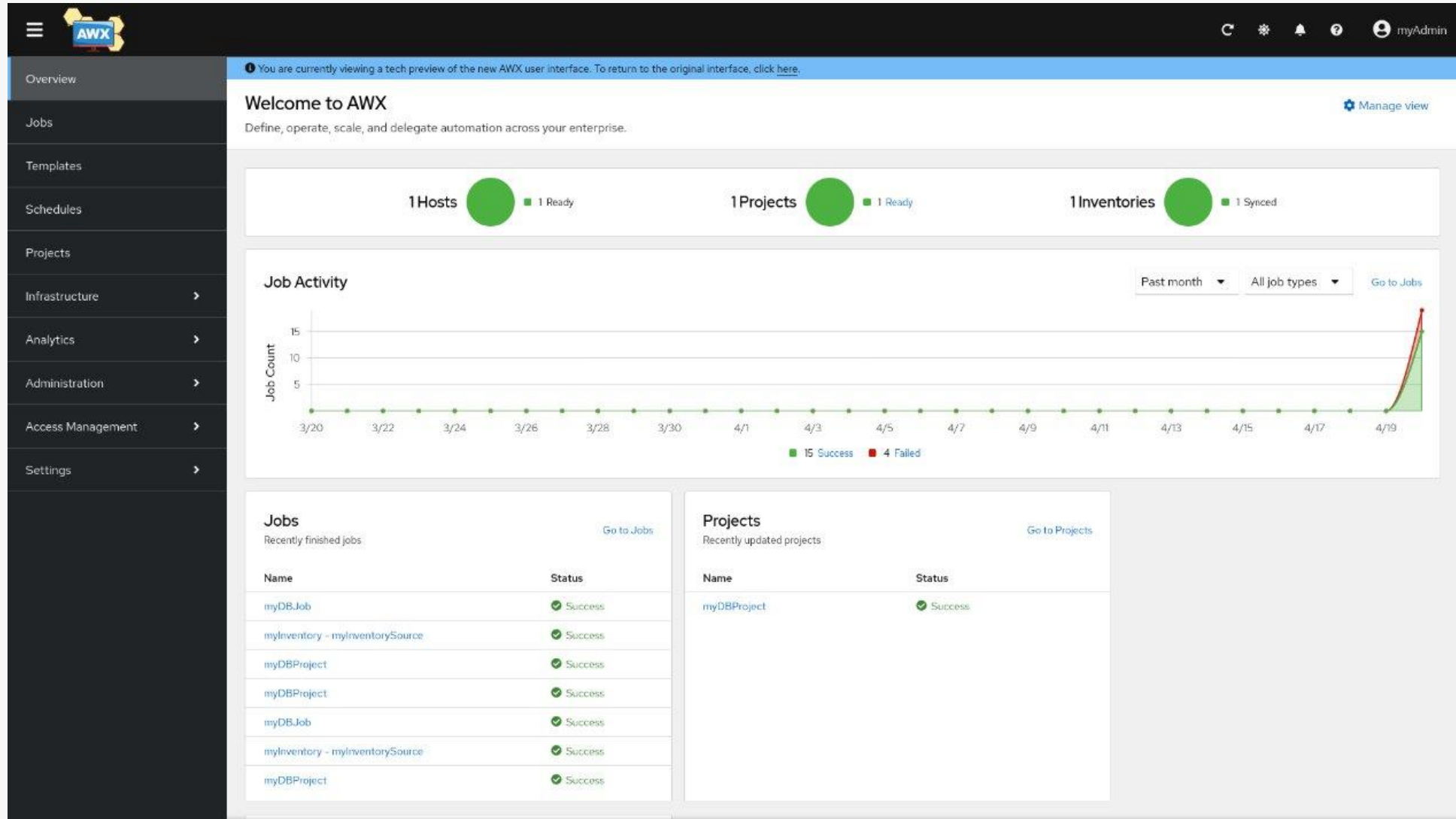
Schedule
automation

AWX

AWX is an open-source that provide Web UI + REST API + Task engine for Ansible

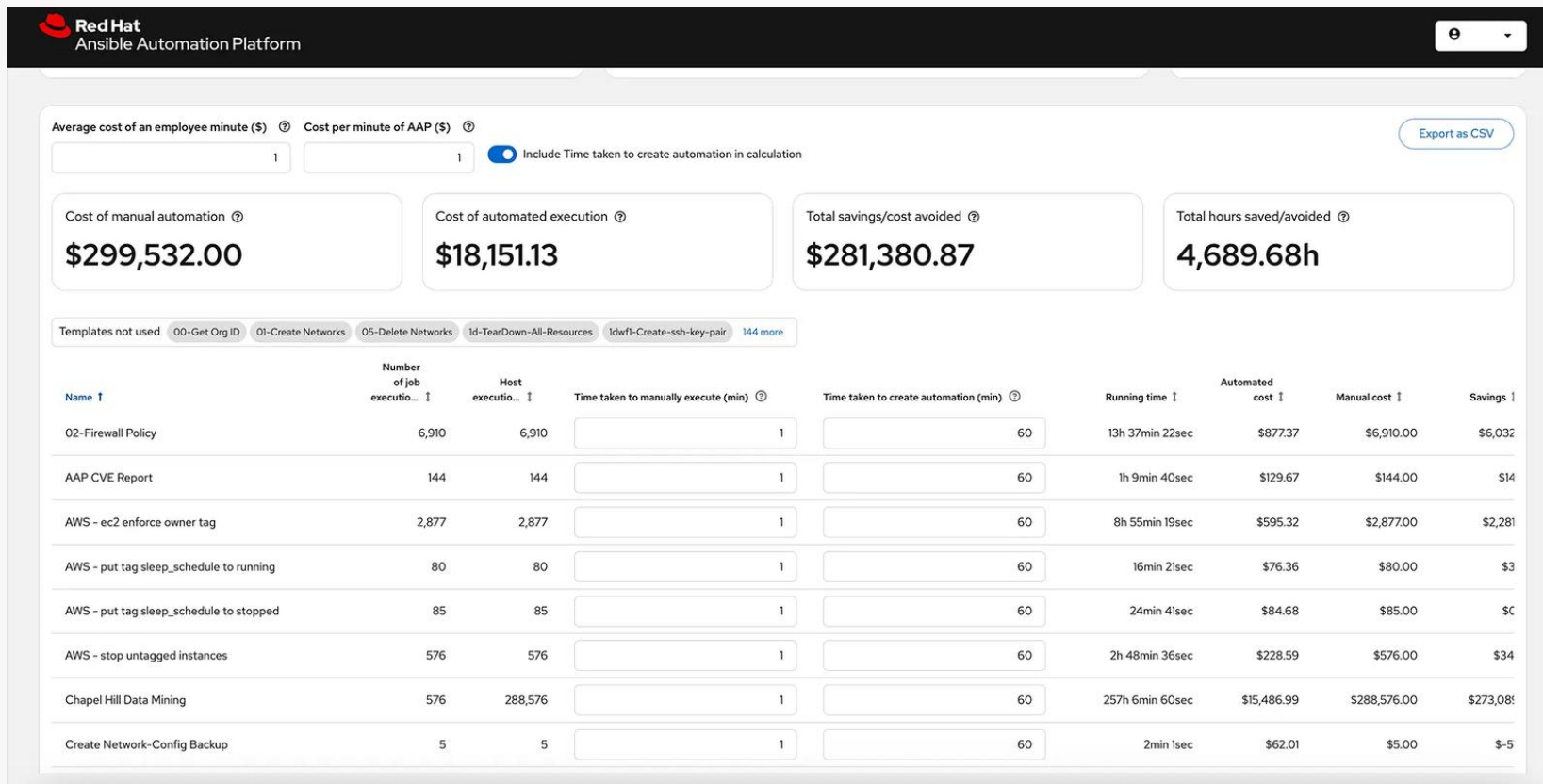


AWX Dashboard



Ansible Automation Platform (AAP)

AAP is not simply AWX with a price tag. It is a broader enterprise automation platform that adds content management, governance, event-driven automation, enterprise support and scalable execution.



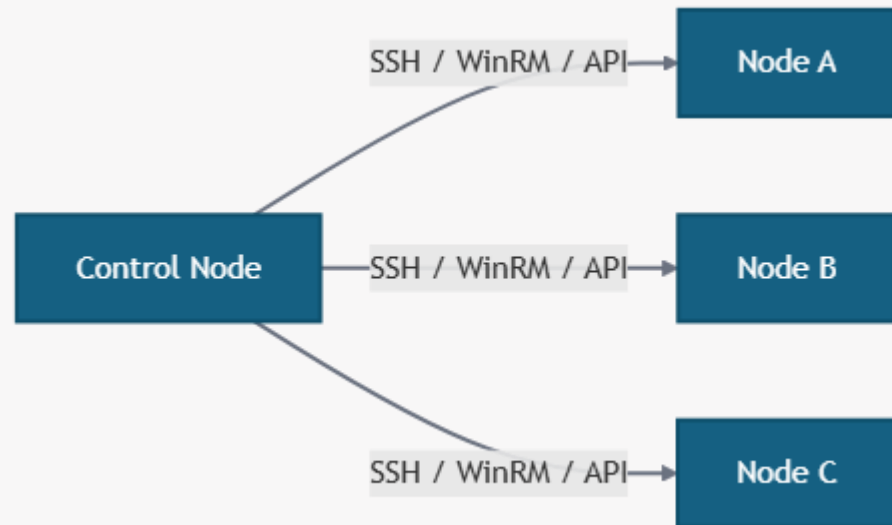
Comparison – Ansible vs Agent-based (Puppet/Chef)



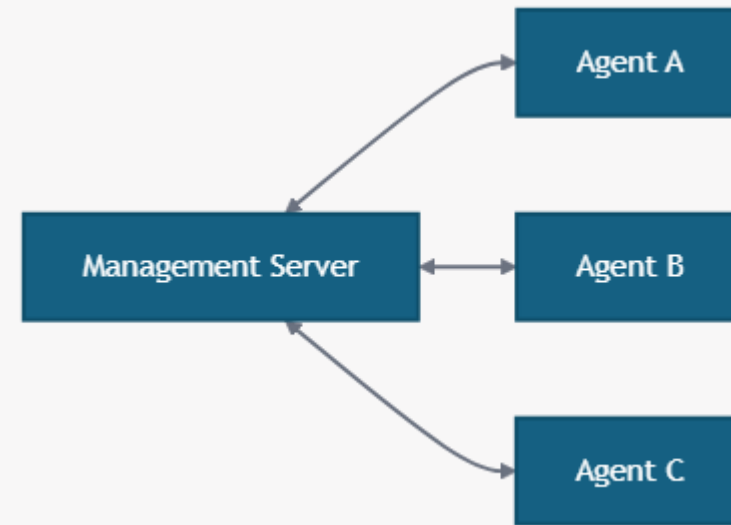
Ansible		vs.	Puppet/Chef	
Agent-less		Architecture		Agent-based
Remote execution from control node		Execution Model		Local agent periodically converges
No persistent agent to maintain		Node Management		Install & maintain agent/client
Provisioning, orchestration, heterogeneous targets		Best Fit		Continuous configuration & compliance

Comparison – Ansible vs Agent-based (Puppet/Chef)

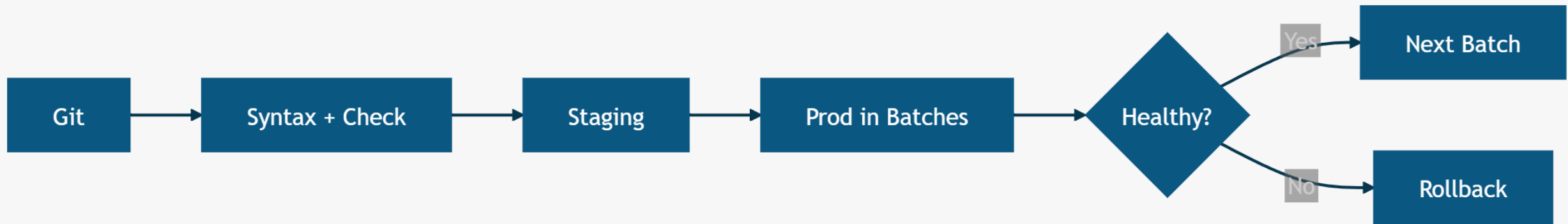
Ansible



Puppet/Chef

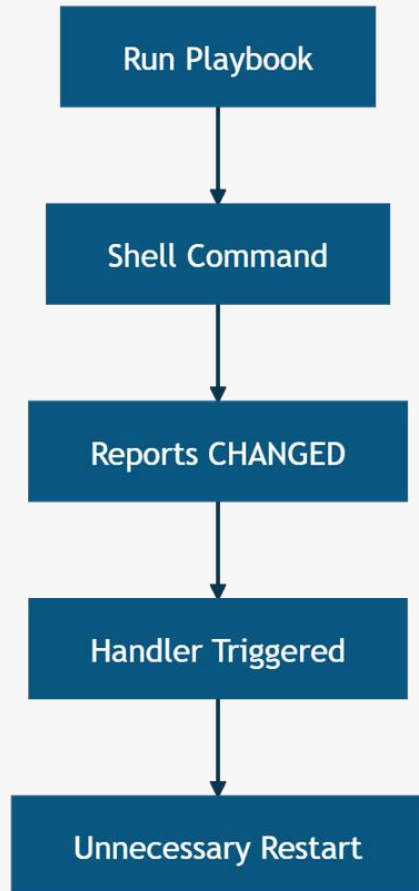


Best Practices - From Playbook to Production



Common Mistake - Treating Ansible Like Bash

Treating Ansible like Bash can break idempotency and automate unnecessary changes across every server.



Mistake

shell: apt-get install -y nginx

Impact

Unnecessary changes / restarts

Better

ansible.builtin.apt

Conclusions

Ansible solves the big DevOps problem of managing many servers consistently without manually SSH-ing into each machine

Pros

- Agentless: Easier adoption
- YAML Playbooks: Readable
- Idempotent modules: Safer repeat runs
- Works across servers, cloud, network devices

Trade-offs

- Slower than agent-based tools at very large scale
- SSH/control node can become bottleneck
- Bad playbooks can spread mistakes quickly
- Requires discipline: inventory, staging, check mode, rollback

References

- [Ansible Community | Ansible documentation](#)
- [Introduction to Ansible - GeeksforGeeks](#)
- [Ansible Roles Best Practices](#)
- [Ansible Tutorial for Beginners](#)



Thank you for listening